

Experiment # 13

Implementation of intercommunication between processes using MPI

OBJECTIVE:

In this lab we are going to learn about Communication between MPI processes using MPI.

SOFTWARE:

- Ubuntu
- mpicc compiler

THEORY:

It is important to observe that when a program running with MPI, all processes use the same compiled binary, and hence all processes are running the exact same code. What in an MPI distinguishes a parallel program running on P processors from the serial version of the code running on P processors? Two things distinguish the parallel program:

- Each process uses its process rank to determine what part of the algorithm instructions are meant for it.
- Processes communicate with each other in order to accomplish the final task.

Even though each process receives an identical copy of the instructions to be executed, this does not imply that all processes will execute the same instructions. Because each process is able to obtain its process rank (using `MPI_Comm_rank`). It can determine which part of the code it is supposed to run. This is accomplished through the use of IF statements. Code that is meant to be run by one particular process should be enclosed within an IF statement, which verifies the process identification number of the process. If the code is not placed within IF statements specific to a particular id, then the code will be executed by all processes.

The second point, communicating between processes; MPI communication can be summed up in the concept of sending and receiving messages. Sending and receiving is done with the following two functions: MPI Send and MPI Recv.

MPI Send

```
int MPI_Send( void* message, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
```

MPI Recv

```
int MPI_Recv( void* message, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status* status)
```

Understanding the Argument Lists

- message - starting address of the send/recv buffer.
- count - number of elements in the send/recv buffer.
- datatype - data type of the elements in the send buffer.
- source - process rank to send the data.
- dest - process rank to receive the data.
- tag - message tag.
- comm - communicator.
- status - status object.

An Example Program:

The following program demonstrate the use of send/receive function in which sender is initialized as node two (2) whereas receiver is assigned as node four (4). The following program requires that it should be accommodated on five (5) nodes otherwise the sender and receiver should be initialized to suitable ranks.

```
#include <iostream.h>

#include <mpi.h>

int main(int argc, char ** argv
{
    int mynode, totalnodes;

    int datasize; // number of data units to be sent/recv

    int sender=2; // process number of the sending process

    int receiver=4; // process number of the receiving process

    int tag; // integer message tag

    MPI_Status status; // variable to contain status
    information

    MPI_Init(&argc,&argv);

    MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);

    MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

    // Determine datasize

    double * databuffer = new double[datasize];

    // Fill in sender, receiver, tag on sender/receiver
```

```

processes,
// and fill in databuffer on the sender process.
if(mynode==sender)

MPI_Send(databuffer, datasize, MPI_DOUBLE, receiver, tag, MPI_COMM_WORLD);

if(mynode==receiver){

    MPI_Recv(databuffer,    datasize,    MPI_DOUBLE,    sender,    tag,    MPI_COMM_WORLD,
    &status);

// Send/Recv complete

MPI_Finalize();

}

```

Key Points:

- In general, the message array for both the sender and receiver should be of the same type and both of same size at least datasize.
- In most cases the sendtype and recvtype are identical.
- The tag can be any integer between 0-32767.
- MPI Recv may use for the tag the wildcard MPI ANY TAG. This allows an MPI Recv to receive from a send using any tag.
- MPI Send cannot use the wildcard MPI ANY TAG. A special tag must be specified.
- MPI Recv may use for the source the wildcard MPI ANY SOURCE. This allows an MPI Recv to receive from a send from any source.
- MPI Send must specify the process rank of the destination. No wildcard exists.

An Example Program: To calculate the sum of given numbers in parallel:

The following program calculates the sum of numbers from 1 to 1000 in a parallel fashion while executing on all the cluster nodes and providing the result at the end on only one node. It should be noted that the print statement for the sum is only executed on the node that is ranked zero (0) otherwise the statement would be printed as much time as the number of nodes in the cluster.

```

#include<iostream.h>

#include<mpi.h>

int main(int argc, char ** argv)

{

```

```

int mynode, totalnodes;

int sum,startval,endval,accum;

MPI_Status status;


MPI_Init(argc,argv);
MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

sum = 0;

startval = 1000*mynode/totalnodes+1;
endval = 1000*(mynode+1)/totalnodes;
for(int i=startval;i<=endval;i=i+1)
sum = sum + i;
if(mynode!=0)
MPI_Send(&sum,1,MPI_INT,0,1,MPI_COMM_WORLD);
else
for(int j=1;j<totalnodes;j=j+1)
{
MPI_Recv(&accum,1,MPI_INT,j,1,MPI_COMM_WORLD,
&status);

sum = sum + accum;
}
if(mynode == 0)
cout << "The sum from 1 to 1000 is: " << sum <<
endl;

MPI_Finalize();
}

```

Lab Task:

Code the above example program that calculates the sum of numbers in parallel on different numbers of nodes. Also calculate the execution time.

[Note: You have to use MPI timer function to calculate execution time also print the time at beginning and end of parallel code segment]

Output: (On Single Node)

Execution Time:

Output: (On Two Nodes)

Execution Time:

Speedup:

Output: (On Four Nodes)

Execution Time:

Speedup:

Output: (On Eight Nodes)

Execution Time:

Speedup: